Building dashboard widgets in MISP 2.5

Typed configuration, scoped data, reusable rendering

CIRCL / MISP Project

https://www.misp-project.org/

Reference snapshot: 29 October 2026



Build one widget, understand the whole path

- Discover and load a custom PHP class.
- Declare typed settings and a shared time filter.
- Query only the signed-in user’s own records.
- Return renderer data and choose a safe cache scope.
- Verify the contract through PHP, HTTP and the browser.

Reference environment

MISP 2.5.50, reviewed 29 October 2026. Patch level and theme matter.

Build one widget, understand the whole path
Discover and load a custom PHP class.
Declare typed settings and a shared time filter.
Query only the signed-in user’s own records.
Return renderer data and choose a safe cache scope.
Verify the contract through PHP, HTTP and the browser.
Reference environment
MISP 2.5.50, reviewed 29 October 2026. Patch level and theme matter.

Allow 35–45 minutes for the presentation and another 25–35 for the lab. Learners should know PHP and basic CakePHP conventions. This course teaches the integrated dashboard, not the separate ZeroMQ application. The README pins the reviewed MISP commit and links to each source contract. Do not imply every reviewed feature shipped in the first 2.5 release.

Understand the model

A widget extends the integrated dashboard

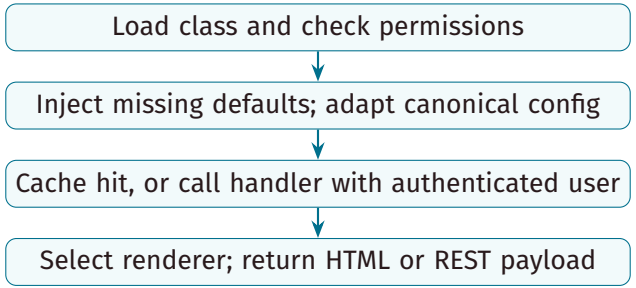
The board owns	Your widget owns
Layout, gallery, configuration, filters, scheduling and export.	Metadata, input validation, data access and renderer-shaped output.

Reuse an existing renderer first. Add custom rendering only when the data needs a different visual form.

A widget extends the integrated dashboard
The board owns
Layout, gallery, configuration, filters, scheduling and export.
Your widget owns
Metadata, input validation, data access and renderer-shaped output.
Reuse an existing renderer first. Add custom rendering only when the data needs a different visual form.

The dashboard is inside MISP at /dashboards. No standalone subscriber or Flask service is involved. Explain that a widget is a PHP class, not a new board framework. The modern board supplies typed configuration, accessible controls and scheduling. Keep external dependencies out of the first exercise so data boundaries are easy to inspect.

The render request has a defined pipeline



Source: DashboardsController::renderWidget

The render request has a defined pipeline
Load class and check permissions
Inject missing defaults; adapt canonical config
Cache hit, or call handler with authenticated user
Select renderer; return HTML or REST payload
Source: DashboardsController::renderWidget

CanonicalTypeAdapter runs before WidgetCache. A cache hit does not execute handler validation or its query; keep cached output valid for the entire cache audience. Permission checks still happen before the cache. The normal web request uses the HTML renderer; JSON callers receive an envelope. The full PHP example follows this pipeline without implementing its own adapter or cache.

Discovery: filename, class and location

```
app/Lib/Dashboard/Custom/
  TrainingLoginSummaryWidget.php

class TrainingLoginSummaryWidget
{
    // Metadata and handler live here.
}
```

- Use a unique class ending in `Widget`.
- Match the filename exactly; no registration list.
- Built-in classes take precedence over Custom.

Discovery: filename, class and location
app/Lib/Dashboard/Custom/
TrainingLoginSummaryWidget.php
class TrainingLoginSummaryWidget
{
// Metadata and handler live here.
}
Use a unique class ending in Widget.
Match the filename exactly; no registration list.
Built-in classes take precedence over Custom.

Dashboard::loadAllWidgets scans built-ins, Custom and one level of custom subdirectories for files ending Widget.php. The loader looks for the built-in class path first. WhoamiWidget already exists, so the new exercise deliberately avoids overriding it. A missing or disallowed class is not a renderer problem. Inspect PHP syntax, worker read permissions and class discovery before debugging the UI.

Metadata gives the gallery a useful entry

Property	Purpose
`title`, `description`	Explain the result and its data scope.
`category`, `thumbnail`	Help discovery; thumbnail is optional.
`render`	Name an existing renderer, here `SimpleList`.
`width`, `height`	Initial tile dimensions.
`schema`	Typed configuration and canonical filter declarations.
`params`, `placeholder`	Advanced option documentation and example JSON.

Metadata gives the gallery a useful entry
Property Purpose
title, description Explain the result and its data scope.
category, thumbnail Help discovery; thumbnail is optional.
render Name an existing renderer, here SimpleList.
width, height Initial tile dimensions.
schema Typed configuration and canonical filter declarations.
params, placeholder Advanced option documentation and example JSON.

These are public PHP properties. This example declares no thumbnail and relies on the renderer preview. The gallery endpoint enriches the older metadata extraction with schema, category and thumbnail. Placeholder content is not a default policy or an access check. A dynamic renderer may implement getRenderer(config); ensure all its possible data shapes match.

Two decisions: who may load it, what may they read?

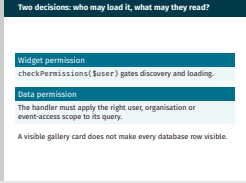
Widget permission

`checkPermissions($user)` gates discovery and loading.

Data permission

The handler must apply the right user, organisation or event-access scope to its query.

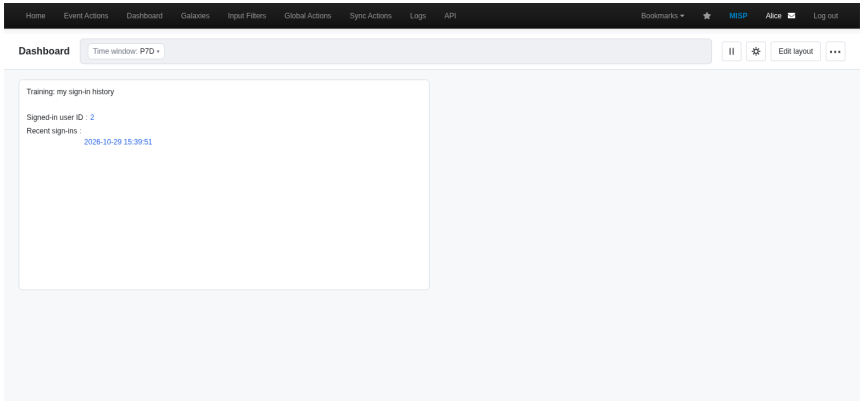
A visible gallery card does not make every database row visible.

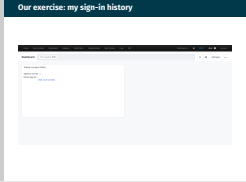


The example accepts an authenticated positive numeric user ID. The handler repeats the identity check for direct invocation, but trusts only the user supplied by the framework. A caller-supplied config.user_id is rejected. For event data, use the relevant ACL-aware MISP retrieval API: arbitrary model find calls are not automatically access-scoped. Some built-in aggregate widgets intentionally report global counts; their semantics cannot be copied blindly to a detailed personal widget.

Build the example

Our exercise: my sign-in history





This is a genuine default-theme browser capture from the reviewed 2.5.50 source. The account is an ordinary User in a synthetic organisation. Its login audit timestamp was created by a real browser login in the lab. The widget shows user ID and sign-in times, not emails, IPs or another user's data. This is a bounded teaching exercise; it is not a proposal to duplicate the existing core Whoami widget.

Declare the canonical time-window setting

```
public $schema = [
    'lookback' => [
        'type' => 'time_window',
        'default' => 'P7D',
        'help' => 'Your own sign-ins; at most 366 days.',
    ],
    // Scalar fields follow in the complete example.
];
```

The shared toolbar follows the declared **type**, not the name `lookback`.

Declare the canonical time-window setting
public $schema = [
'lookback' => [
'type' => 'time_window',
'default' => 'P7D',
'help' => 'Your own sign-ins; at most 366 days.',
],
// Scalar fields follow in the complete example.
];
The shared toolbar follows the declared type, not the name lookback.

This abbreviated schema excerpt keeps the contract visible on screen. The complete file has a fuller help string. The canonical declaration enables the typed configuration control and toolbar reachability. Choosing a parameter name such as lookback is valid; time_window does not have to be the key name. Do not declare a canonical type unless the handler can consume its adapter output.

Canonical inputs and scalar settings serve different jobs

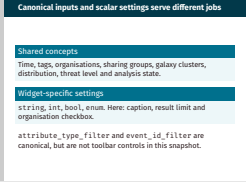
Shared concepts

Time, tags, organisations, sharing groups, galaxy clusters, distribution, threat level and analysis state.

Widget-specific settings

`string`, `int`, `bool`, `enum`. Here: caption, result limit and organisation checkbox.

`attribute_type_filter` and `event_id_filter` are canonical, but are not toolbar controls in this snapshot.



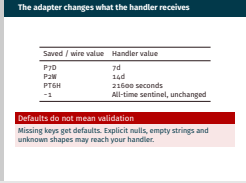
WidgetSchema defines twelve canonical types, ten of which are toolbar eligible. The README lists all names, including org_meta_filter and date_range. Scalar types do not create toolbar chips. Enum declarations require an enum list and can provide enum_labels. A shared toolbar change affects only widgets that declare the relevant canonical type. Avoid inventing universal tag-filter syntax: widget contracts still differ.

The adapter changes what the handler receives

Saved / wire value	Handler value
P7D	7d
P2W	14d
PT6H	21600 seconds
-1	All-time sentinel, unchanged

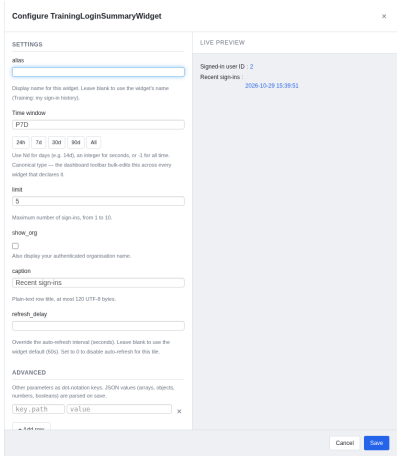
Defaults do not mean validation

Missing keys get defaults. Explicit nulls, empty strings and unknown shapes may reach your handler.



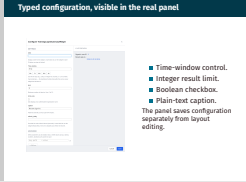
The verifier invokes the real CanonicalTypeAdapter before the example handler. Legacy day strings and raw seconds are preserved. The adapter is deliberately compatibility-oriented, so validate its resulting values. This example accepts positive seconds up to 366 days, bounded day strings and minus one; rejects zero, booleans, null, arrays, oversized windows and unknown strings. Other canonical types have their own translations and validation; inspect those before adopting them.

Typed configuration, visible in the real panel



- Time-window control.
- Integer result limit.
- Boolean checkbox.
- Plain-text caption.

The panel saves configuration separately from layout editing.



Use a live demonstration or the larger README image to inspect the field labels. The capture includes framework-managed alias and refresh_delay fields and an advanced section. They are not extra SQL parameters. The browser test changed the caption to HTML-like text, enabled the checkbox, saved successfully and confirmed literal rendering. Do not present a screenshot alone as proof of persistence or validation.

Validate before querying

- `limit`: integer from 1 to 10.
- `show_org`: real boolean, not the string `"false"`.
- `caption`: nonempty UTF-8, at most 120 bytes, no controls.
- Reject unknown keys, including a supplied `user_id`.

Framework-owned settings

Accept `alias` and `refresh_delay`; do not let them change data.

Validate before querying
limit: integer from 1 to 10.
show_org: real boolean, not the string "false".
caption: nonempty UTF-8, at most 120 bytes, no controls.
Reject unknown keys, including a supplied user_id.
Framework-owned settings
Accept alias and refresh_delay; do not let them change data.

BadRequestException yields HTTP 400 for these handler failures in the live test. Do not cast strings to booleans: the nonempty string false is truthy in PHP. The example uses strict is_int and is_bool checks and array_key_exists so explicit null does not silently become a default. This is one possible public contract; compatible existing widgets may need a documented migration path instead of immediate strict rejection.

Apply identity in the actual SQL conditions

```
$conditions = [
    'Log.user_id' => (int)$user['id'],
    'Log.action' => 'login',
];
if ($seconds !== -1) {
    $conditions['Log.created >='] =
        date('Y-m-d H:i:s', time() - $seconds);
}
```

Query identity comes from the authenticated user, never configuration.

Apply identity in the actual SQL conditions

```
$conditions = [
    'Log.user_id' => (int)$user['id'],
    'Log.action' => 'login',
];
if ($seconds !== -1) {
    $conditions['Log.created >='] =
        date('Y-m-d H:i:s', time() - $seconds);
}
```

Query identity comes from the authenticated user, never configuration.

This is the data boundary. Date formatting follows Log::beforeSave's use of date in the application's PHP timezone. The handler also bounds output with a selected field list, recursive minus one and limit 1–10. Ordering is created descending and then ID descending for stable ties. A limit caps returned rows; it is not proof that a query is inexpensive. For larger audit tables, inspect query plans and indexes rather than adding broad unbounded scans to frequently refreshed widgets.

Return the renderer’s data, not a page

```
$rows = [
    ['title' => 'Signed-in user ID',
     'value' => (string)$user['id']],
    ['title' => $caption,
     'value' => $times ?: 'No matching sign-ins.'],
];
return $rows;
```

`SimpleList` accepts scalar or array values.
Empty history is a valid result. A database failure is an error.

Return the renderer’s data, not a page
$rows = [
['title' => 'Signed-in user ID',
'value' => (string)$user['id']],
['title' => $caption,
'value' => $times ?: 'No matching sign-ins.'],
];
return $rows;
SimpleList accepts scalar or array values.
Empty history is a valid result. A database failure is an error.

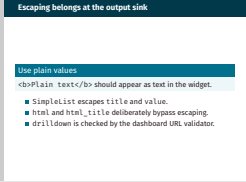
The snippet omits the optional organisation row for readability. The real template escapes title and value and joins array values with line breaks. The example returns plain timestamps, not HTML. It deliberately does not catch database exceptions and return an empty array, because that would hide a failure as no activity. Test empty data and backend failure independently.

Escaping belongs at the output sink

Use plain values

`<b>Plain text</b>` should appear as text in the widget.

- `SimpleList` escapes `title` and `value`.
- `html` and `html_title` deliberately bypass escaping.
- `drilldown` is checked by the dashboard URL validator.



Do not pre-escape text, which causes double escaping in the real renderer. Avoid raw HTML fields for user-controlled content. URL validation drops dangerous schemes and arbitrary off-host URLs; permitted origins include the MISP origin and the configured CVE reference origin. Test the actual PHP element as well as JSON. The committed verifier includes the real SimpleList template with actual CakePHP h() rather than reproducing its rendering logic in a fake helper.

Refresh interval and cache TTL are different

Example property	Meaning
`autoRefreshDelay = 60`	Request a refresh every 60 seconds
`cache_duration = 30`	Reuse a result for 30 seconds
`cache_scope = 'user'`	Separate cached results by user

Manual refresh can return a cache hit. Redis failure falls back to live work.

Refresh interval and cache TTL are different
Example property | Meaning
autoRefreshDelay = 60 | Request a refresh every 60 seconds
cache_duration = 30 | Reuse a result for 30 seconds
cache_scope = 'user' | Separate cached results by user
Manual refresh can return a cache hit. Redis failure falls back to live work.

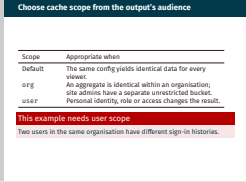
These are public properties with dollar prefixes in PHP. The legacy cacheLifetime property taught in the earlier lesson does not opt into the generic cache in this reviewed controller. Positive cache_duration enables it; zero disables it. An optional cache_path changes the prefix. The default prefix is derived from the class name. During debugging, use a disabled cache or wait for expiry before drawing conclusions about code changes; do not recommend flushing a shared Redis database.

Choose cache scope from the output's audience

Scope	Appropriate when
Default	The same config yields identical data for every viewer.
`org`	An aggregate is identical within an organisation; site admins have a separate unrestricted bucket.
`user`	Personal identity, role or access changes the result.

This example needs user scope

Two users in the same organisation have different sign-in histories.



A private SQL query followed by a shared cache can still leak data. Permissions and cache audience are separate decisions. WidgetCache keys the translated config, sorted at top level, and excludes alias and refresh_delay. Thus the handler must not make its data depend on those fields. The verifier checks real helper keys for same-org users; the integration test exercises A/B/A through actual HTTP and Redis. Missing usable identity causes scoped remember to run live rather than cache in a shared bucket.

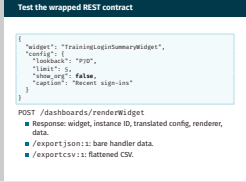
Verify and contribute

Test the wrapped REST contract

```
{
  "widget": "TrainingLoginSummaryWidget",
  "config": {
    "lookback": "P7D",
    "limit": 5,
    "show_org": false,
    "caption": "Recent sign-ins"
  }
}
```

`POST /dashboards/renderWidget`

- Response: widget, instance ID, translated config, renderer, data.
- `/exportjson:1`: bare handler data.
- `/exportcsv:1`: flattened CSV.



The README gives a curl command with a locally provided API key, JSON content type and Accept header. Never commit a real key or turn off TLS validation. The controller accepts config as an object or JSON string; teach the object form. The response config uses 7d after adaptation, unlike the saved canonical P7D. A successful REST payload proves neither safe HTML rendering nor a functioning JavaScript chart. Keep normal CSRF handling in browser requests.

A layout identifies instances, not just classes

```
[
  {
    "instance_id": "training_signins",
    "widget": "TrainingLoginSummaryWidget",
    "config": {"lookback": "P7D", "limit": 5},
    "position": {"x": 0, "y": 0, "w": 6, "h": 4}
  }
]
```

Keep instance IDs stable. Importing a layout changes the personal board.

A layout identifies instances, not just classes
Keep instance IDs stable. Importing a layout changes the personal board.

This shortened illustration omits optional example config keys; schema defaults supply them. The complete importable layout is in examples. Multiple instances of a class can have different IDs and settings. Older width/height-only layouts have adaptation paths, but new examples should use position x/y/w/h. GET /dashboards returns the REST layout; /dashboards/export instead wraps saved UserSetting data. Import and template selection never grant access to underlying records.

Saving layout does not save every kind of change

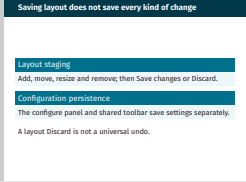
Layout staging

Add, move, resize and remove; then Save changes or Discard.

Configuration persistence

The configure panel and shared toolbar save settings separately.

A layout Discard is not a universal undo.



Controller updateSettings saves the personal UserSetting dashboard layout. updateWidgetSettings patches configurations separately. Shared templates are Dashboard records. Learners should export a board before experimenting, and save settings, reload and inspect server state rather than assuming an in-memory preview means persistence. This separation also matters when writing integration tests.

Three layers of evidence

Layer	Verify
PHP contracts	Real schema, adapter, cache keys and template; recorded database boundary.
Live HTTP	Actual SQL results, user A/B/A, invalid inputs, export and layout import/readback.
Browser	Discovery, typed settings, saved state, literal caption, resize and usable controls.

The supplied verifier runs **59 checks**; it is not an integration test.

Three layers of evidence
Layer Verify
PHP contracts Real schema, adapter, cache keys and template; recorded database boundary.
Live HTTP Actual SQL results, user A/B/A, invalid inputs, export and layout import/readback.
Browser Discovery, typed settings, saved state, literal caption, resize and usable controls.
The supplied verifier runs **59 checks**; it is not an integration test.

The standalone verifier loads actual CakePHP helpers and MISP classes from a provided installation. Only Log::find is replaced with an in-memory recorder; it tests query arguments and failure propagation. It does not access SQL, Redis or HTTP. The real lab separately verified two non-admin users in the same org, genuine logins, rejection cases, wrapped/bare JSON, import/readback and browser config saves. Be precise about evidence rather than describing a mock query as a tested database.

Workshop: install, challenge, remove

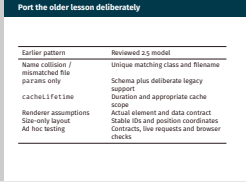
1. Export your board; install the one custom PHP file.
2. Lint it and run `verify_widget.php` against MISP.
3. Add the widget; save a caption and checkbox setting.
4. Render as A, B, A within the cache TTL.
5. Try an invalid boolean, limit, null and supplied user ID.
6. Restore the board and remove the training PHP file.

Workshop: install, challenge, remove
1. Export your board; install the one custom PHP file.
2. Lint it and run verify_widget.php against MISP.
3. Add the widget; save a caption and checkbox setting.
4. Render as A, B, A within the cache TTL.
5. Try an invalid boolean, limit, null and supplied user ID.
6. Restore the board and remove the training PHP file.

Use two disposable ordinary users in the same organisation and sign in through the browser to generate Log.action login rows. The README includes exact installation and request commands. An API key requires API permission but no site-admin role for this widget. Installation requires operator access to the custom directory. Check permissions if the gallery entry is absent. Removing a class while leaving its instance in a board creates a missing-widget state, so clean the layout first.

Port the older lesson deliberately

Earlier pattern	Reviewed 2.5 model
Name collision / mismatched file	Unique matching class and filename
`params` only	Schema plus deliberate legacy support
`cacheLifetime`	Duration and appropriate cache scope
Renderer assumptions	Actual element and data contract
Size-only layout	Stable IDs and position coordinates
Ad hoc testing	Contracts, live requests and browser checks



The original lesson, assets and PDF are preserved in the archive and excluded from standard training. Migrate saved configs intentionally: adding a schema does not rewrite them or secure an old query. Preserve compatibility where it matters and document deliberate changes. Cache invalidation after changing output semantics needs an operator plan; a temporary cache disable or new dedicated prefix is safer than a broad shared-cache flush.

If a new renderer is necessary

- Start with the PHP element's precise data shape.
- Reuse theme tokens and accessible, responsive markup.
- Extend the existing dashboard chart modules where needed.
- Add a gallery preview for the renderer.
- Test empty data, failures, resizing, keyboard and themes.

If a new renderer is necessary
- Start with the PHP element's precise data shape.
- Reuse theme tokens and accessible, responsive markup.
- Extend the existing dashboard chart modules where needed.
- Add a gallery preview for the renderer.
- Test empty data, failures, resizing, keyboard and themes.

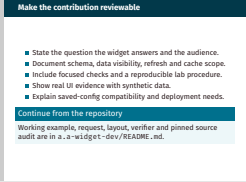
PHP renderer elements are in app/View/Elements/dashboard/Widgets. JavaScript lives under app/webroot/js/dashboard, with chart modules under charts and preview glyph handling at gallery/renderthumbs.mjs. Reuse the deployed assets and framework instead of dropping in replacement vendor libraries or inline scripts. For SimpleList no JavaScript renderer is needed. A new renderer should come with evidence that HTML and chart initialization work, not just a screenshot of JSON.

Make the contribution reviewable

- State the question the widget answers and the audience.
- Document schema, data visibility, refresh and cache scope.
- Include focused checks and a reproducible lab procedure.
- Show real UI evidence with synthetic data.
- Explain saved-config compatibility and deployment needs.

Continue from the repository

Working example, request, layout, verifier and pinned source audit are in `a.a-widget-dev/README.md`.



Review MISP's current contribution guidance before proposing a core widget. Custom deployments can keep a local class without upstreaming it. This example intentionally duplicates a simple personal-status concept for teaching, so do not submit it as a new core feature unchanged. Reviewers should be able to reproduce data-boundary and cache-isolation checks without a production dataset or a committed credential.